

Sesión 1

Web Scraping

Programación para Ciberseguridad — Fase III

Python Ofensivo y Defensivo — Semana 8

Objetivo de la sesión

- Entender lo mínimo de HTTP para automatizar peticiones web
- Hacer peticiones con requests y leer su respuesta
- Extraer datos de HTML con BeautifulSoup
- Conocer Scrapy como framework para scraping a mayor escala

HTTP: lo mínimo indispensable

Una petición HTTP tiene un **método** (GET, POST), una **URL**, y recibe una **respuesta** con un **código de estado**.

Código	Significado
200	Éxito
403	Prohibido
404	No encontrado
429	Demasiadas peticiones

La librería requests

```
import requests

respuesta = requests.get("https://example.com")
print(respuesta.status_code)
print(respuesta.text[:200])

parametros = {"q": "ciberseguridad"}
r = requests.get("https://example.com/buscar", params=parametros)
```

Tip

Siempre revisa `status_code` antes de procesar el contenido — un 403 o 429 significa que el sitio está bloqueando tus peticiones.

BeautifulSoup: extraer datos del HTML

```
from bs4 import BeautifulSoup

html = requests.get("https://example.com").text
sopa = BeautifulSoup(html, "html.parser")

titulo = sopa.title.string
enlaces = sopa.find_all("a")

for enlace in enlaces:
    print(enlace.get("href"), enlace.text)
```

Error común

El HTML de un sitio cambia

Un scraper que funciona hoy puede romperse mañana si el sitio actualiza su diseño. Siempre envuelve cada `find()` en manejo de errores — puede devolver `None`.

Scrapy: scraping a escala

```
import scrapy

class MiSpider(scrapy.Spider):
    name = "mi_spider"
    start_urls = ["https://example.com"]

    def parse(self, response):
        for enlace in response.css("a::attr(href)").getall():
            yield {"url": enlace}
```

Scrapy maneja peticiones concurrentes, reintentos, y respeta robots.txt automáticamente — útil para cientos o miles de páginas.

Aplicación en ciberseguridad

OSINT: Open Source Intelligence

Recolectar información pública de una organización (tecnologías usadas, empleados, subdominios mencionados en documentos) sin tocar directamente su infraestructura — reconocimiento completamente pasivo.

Ética desde el inicio

Antes de scrapear cualquier sitio

Revisa los Términos de Servicio, el archivo `robots.txt`, y si tienes autorización explícita para el objetivo que investigas. Recolectar información pública no siempre es legal ni ético solo por estar disponible.

Cierre de la sesión

1. `requests` hace peticiones HTTP; siempre revisa `status_code`
2. `BeautifulSoup` convierte HTML en algo navegable: `find()`, `find_all()`
3. `Scrapy` es el siguiente paso para scraping a mayor escala
4. Todo esto es OSINT — reconocimiento pasivo, con límites éticos claros

Práctica: R21 (scraper OSINT guiado) y R22 (TryHackMe: OhSINT)